



ISYS3476 (Postgraduate)

Managing Semi-structured and Unstructured Data

Assignment 1 Part A: Pre-processing & Feature Engineering

Assessment	Assignment 1 Part A
Type	Individual
Weighting	20%
Due Date	Sunday, 16 August 2026, 11:59 pm (Melbourne time)
Submission	Canvas ZIP
Late Penalty	10% of the total available marks per calendar day or part thereof. Submissions more than five calendar days late will not be marked unless an approved extension, equitable assessment arrangement, or special consideration applies.
Total Points	20 points

Starter code and final submission

Starter code: Download and unzip the provided starter ZIP from Canvas. Before starting the tasks, follow `README.md` in the starter folder to set up Python and install the packages in `requirements.txt`.

Final submission: Submit one completed assignment ZIP on Canvas. The Canvas ZIP is the only source used for marking.

What to read. This **Main Spec** contains everything you *must* implement and submit, plus minimal examples. Detailed marking rubrics are in `partA_rubric_PG.pdf`.

1 Overview

You will implement four core feature-engineering components used for indexing and ranking:

- **Task 1 (6 pts):** robust HTML-to-tokens preprocessing (`preprocess`)
→ **Retrieval-based performance (4 pts) + Procedure docstring (2 pts)**
- **Task 2 (4 pts):** character n-grams and position mappings
- **Task 3 (3 pts):** TF-IDF variants (`log` and `bm25`)
- **Task 4 (3 pts):** semantic vector aggregation (`tfidf_weighted` and `meanmax`)
- **Engineering development evidence (2 pts):** assessed from `DEVELOPMENT.md`

- **Code quality across all tasks (2 pts):** assessed *holistically* across your Part A code (readability, robustness, documentation, etc.)

Starter code and fixed interfaces

You will be given starter code with fixed function signatures. **Do not change** function names, parameter names, default values, return types, or file locations. You may add helper functions/files, but the required interfaces must remain unchanged. Changes can break the autograder.

Sanity check (before submission)

Run `python -m utils.check_submission_partA` before submitting.

- It checks that Part A imports and runs, checks basic interface/contract integrity, and warns about common disallowed packages.
- Task 1 reports both dev profiles and their combined Average Hit@K. Task 2 uses public examples. Tasks 3–4 check output shape/format.
- Passing does not predict full marks. Use it as a runnable-interface check.

2 Task 1: Document Pre-processing Pipeline (6 points)

2.1 What you implement

Implement `preprocess` in the starter code.

Function contract.

```
def preprocess(raw_html_list: List[str]) -> List[List[str]]:
    """Clean and tokenize a batch of noisy documents.
    Args:
        raw_html_list: One raw HTML or noisy web-text string per document.
    Returns:
        One token list per input document, preserving input order and length.
        Produce the final tokens with nltk.word_tokenize after cleaning. Return
        an empty list for a document with no usable text.

    Preprocessing guidance:
        Convert noisy HTML or web text into cleaned, human-readable text.
        Inspect both public dev profiles yourself, then choose cleaning steps
        for the noise patterns you actually find there.
    """
```

2.2 Task 1 retrieval-based assessment (4 points)

The goal of Task 1 is to produce tokens that support reliable lexical retrieval across different surface noise. The public profiles contain the same underlying cases with different noise. Inspect both and choose preprocessing that remains effective across them.

Public examples are in `data/dev_profile_a` (Profile A) and `data/dev_profile_b` (Profile B). The sanity checker reports each profile and their combined Average Hit@K. You do **not** implement the ranker. Improve the tokens produced by `preprocess`.

For the marked hidden test:

- Hidden documents and queries differ from public dev. Each case is presented and evaluated under both profile categories.
- The combined metric averages the two profile observations.

- Documents are ranked using the Week 02-style token-overlap ranker: score = number of unique query tokens also appearing in the document tokens. Token matching is exact and case-sensitive. Any normalisation must be produced by `preprocess`.
- Results are sorted by score descending, with deterministic tie-breaking by document ID. The final metric is the average of Hit@1 and Hit@5 across all hidden queries.

See `utils/check_submission_partA.py` if you want to inspect the public dev-set sanity check.

2.3 Required preprocessing procedure docstring (2 points)

In addition to the retrieval-based assessment, you must write a short **procedure description** in the docstring of your `preprocess` function.

- Explain how your submitted implementation turns the provided noisy documents into cleaner text before tokenization, including why the main operation order matters.
- Ground your choices in concrete observations from both public profiles and explain why they should generalise beyond individual dev examples.
- The explanation must be consistent with the submitted code. Unsupported or invented steps will not receive credit.

2.4 Example

```
# Input
raw_html_list = [
    "<html><body><h1>Machine Learning</h1><p>AI & deep learning are important.</p></body></html>",
    "<div>Natural language processing is fascinating!</div>",
    "Great!!!</div>",
]

# Illustrative Output (one acceptable result)
[
    ['Machine', 'Learning', 'AI', 'and', 'deep', 'learning', 'are', 'important', '.'],
    ['Natural', 'language', 'processing', 'is', 'fascinating', '!'],
    ["Great", "!"],
]
```

3 Task 2: Character N-grams and Positions (4 points)

Implement the following functions as specified in the starter code.

Function contracts.

```
def make_ngrams_chars(text: str, n: int) -> List[str]:
    """Build character n-grams from boundary-padded text.
    Args:
        text: Source string from which to build character n-grams.
        n: Number of characters in each n-gram.
    Returns:
        Character n-grams from '$' + text + '$' in left-to-right order.
        Return an empty list when the padded text is shorter than n.
    """

def make_positions(tokens: List[str]) -> Dict[str, List[int]]:
    """Map each token to its positions in a token sequence.
    Args:
        tokens: Token sequence in document order.
```

```
Returns:
    A mapping from each token to its 0-based positions in tokens.
    """
```

3.1 Examples

3.1.1 Character n-grams (\$ is a boundary marker)

```
make_ngrams_chars("hello", 3)
# -> ['$he', 'hel', 'ell', 'llo', 'lo$']
```

3.1.2 Positions (0-based)

```
tokens = ["the", "quick", "brown", "fox", "jumps", "over", "the", "dog"]
make_positions(tokens)
# -> {'the': [0, 6], 'quick': [1], 'brown': [2], 'fox': [3],
#     'jumps': [4], 'over': [5], 'dog': [7]}
```

4 Task 3: Advanced TF-IDF Variants (3 points)

4.1 What you implement

Implement `tfidf_variants` in the starter code.

Function contract.

```
def tfidf_variants(docs: List[List[str]], tf_mode: str = 'len', k: float = 1.2)
    -> Tuple[np.ndarray, Dict[str, int]]:
    """Build a TF-IDF document-term matrix.
    Args:
        docs: One token list per document.
        tf_mode: Term-frequency mode, one of 'len', 'log', or 'bm25'.
            The 'log' mode uses the natural logarithm.
        k: BM25 term-frequency saturation parameter.
    Returns:
        A pair (X, vocab). X has one row per document and one column per
        vocabulary term. vocab contains every unique input term and assigns
        columns in alphabetical term order.
    """
```

Assessed `tf_mode`: `log` and `bm25`. `len` is required for Task 4.

4.2 Required equations

TF-IDF definition

$$\text{TF-IDF}(t, d) = \text{TF}(t, d) \times \text{IDF}(t) = \text{TF}(t, d) \times \ln \left(\frac{N}{\text{df}(t)} \right) \quad (1)$$

where N is the total number of documents and $\text{df}(t)$ is the document frequency of term t . **IDF is unsmoothed** (i.e., no $+1$ terms).

Notation note. In these equations, $\text{count}(t, d)$ denotes the *unweighted count* of term t in document d , while $\text{TF}(t, d)$ denotes the *term-frequency weighting* used inside TF-IDF.

Length-normalised TF ('len') (required for Task 4, covered in Week 03 workshop)

Uses length-normalised term frequency where $|d|$ is the document length:

$$\text{TF}_{\text{len}}(t, d) = \frac{\text{count}(t, d)}{|d|} \quad (2)$$

Log-normalised TF (natural log, base e)

$$\text{TF}_{\log}(t, d) = \begin{cases} 1 + \ln(\text{count}(t, d)) & \text{if } \text{count}(t, d) > 0 \\ 0 & \text{otherwise} \end{cases} \quad (3)$$

BM25-style TF (bounded saturation)

$$\text{TF}_{\text{bm25}}(t, d) = \frac{(k + 1) \times \text{count}(t, d)}{k + \text{count}(t, d)} \quad (4)$$

with default $k = 1.2$.

4.3 Example

```
docs = [['cat', 'sat', 'mat'], ['cat', 'cat', 'dog']]
tfidf_matrix, vocab = tfidf_variants(docs, tf_mode='log')
# Returns: (array([[...], [...]]), {'cat': 0, 'dog': 1, 'mat': 2, 'sat': 3})
```

5 Task 4: Semantic Vector Aggregation (3 points)

5.1 What you implement

Implement `semantic_vector` in the starter code.

Function contract.

```
def semantic_vector(docs: List[List[str]], method: str = 'meanmax') -> np.
    ndarray:
        """Build one semantic vector per document.
        Args:
            docs: One token list per document.
            method: Aggregation method, either 'tfidf_weighted' or 'meanmax'.
        Returns:
            A matrix with shape (N, D) for 'tfidf_weighted' or (N, 2D) for
            'meanmax', where N is the number of documents and D is the embedding
            dimension.
        """
```

Assessed method: `tfidf_weighted` and `meanmax`. For both assessed methods, treat each out-of-vocabulary token as `<unk>`.

5.2 Required equations

TF-IDF weighted average (`tfidf_weighted`) Use the special token `<unk>` for out-of-vocabulary tokens (as specified above). **TF-IDF must use 'len' mode for consistent marking.** Let $\text{doc}_i = (w_{i,1}, \dots, w_{i,n_i})$ be the token sequence (list).

$$\vec{d}_i = \frac{\sum_{j=1}^{n_i} \text{tfidf}(w_{i,j}, \text{doc}_i) \times \vec{e}_{w_{i,j}}}{\sum_{j=1}^{n_i} \text{tfidf}(w_{i,j}, \text{doc}_i)} \quad (5)$$

Mean-Max concatenation (`meanmax`)

$$\vec{d}_i = \left[\frac{1}{n_i} \sum_{w \in \text{doc}_i} \vec{e}_w \right] \oplus \left[\max_{w \in \text{doc}_i} (\vec{e}_w) \right] \quad (6)$$

where $\oplus$ denotes concatenation (output dimension $2 \times$ embedding dimension).

5.3 Embedding resource constraint

You *must* use the provided embedding dictionary and dimensions from the starter code (i.e., `_WORD_VEC` and `_DIM`). Do not use external embeddings or extra ML libraries.

5.4 Example

```
docs = [['cat', 'dog'], ['animal', 'pet', 'cat']]
vecs = semantic_vector(docs, method='tfidf_weighted')
# -> numpy array with shape [2, embedding_dim]

vecs = semantic_vector(docs, method='meanmax')
# -> numpy array with shape [2, 2 * embedding_dim]
```

6 Engineering Development Evidence (2 points)

Complete `DEVELOPMENT.md` with two separate excerpts: one failed self-check output and one implementation error or incorrect output. For each, state what you changed and show the rerun result. This assesses engineering development evidence, not method design or experiments.

7 Overall Code Quality Standards

Code quality is assessed *holistically* across all Part A tasks (2 points). We look for:

- **Clarity:** readable structure, meaningful names, concise docstrings/comments, and helper functions where useful.
- **Defensive programming:** handles invalid or unexpected inputs sensibly rather than crashing.
- **Compliance and cleanliness:** preserves interfaces/layout, keeps agent instructions/config files unchanged, uses only permitted imports, avoids environment hacks, and removes unused code.

8 Submission Requirements

8.1 Allowed libraries

Use **Python 3.10 to 3.14 inclusive**. Dependencies are listed in `requirements.txt`. Permitted imports are `numpy`, `nltk`, `bs4` (BeautifulSoup), and the Python standard library. Other ML/NLP libraries are not permitted.

8.2 What to submit

- **Submit:** one ZIP on Canvas. Keep the folder structure unchanged and include the unchanged `AGENTS.md` and completed `DEVELOPMENT.md`.
- **ZIP naming:** use `YourName_s123456.zip` (e.g., `ZhuangLi_s123456.zip`).
- **Marking source:** only the ZIP submitted on Canvas is marked.
- **Do not submit virtual environments** such as `.venv/`, `env/`, `venv/`, or `.conda/`.

AI assistance policy. Do not use AI tools to generate a full solution. Keep the matching instruction in `AGENTS.md` unchanged.